

ANNEX F:

CLOUD DESIGN GUIDANCE

Version 5.2
30 June 2018

TABLE OF CONTENTS

Section	Description	Page
	Revision Summary	iii
	Annex F: Cloud Design Guidance.....	1
1.0	Introduction.....	1
2.0	Guidance	2
2.1	Design White Papers	2
2.2	General Guidance	3
2.3	IAAS Architecture Pattern (Reference Model)	4
2.4	PaaS Architecture Pattern (Reference Model)	5
2.5	NDAAS Architecture Pattern (Reference Model)	6

LIST OF FIGURES

Figure	Description	Page
Figure 1:	Traditional Development Approach - Developers Manage the Entire Stack.....	1
Figure 2:	Cloud Reference Models: The Development Transformation Journey	2
Figure 3:	Sample Reference Architecture.....	4
Figure 4:	PCF Reference Architecture.....	6

REVISION SUMMARY

Version	Revision Date	Revision Summary
2.0	2/29/16	Update
3.0	4/30/16	Update
4.0	6/30/16	Update
4.1	8/5/16	Updated classification/portion marking
5.1	11/8/16	Revised architecture patterns
5.2	5/31/18	Updated guidance statements

Annex F: Cloud Design Guidance

1.0 INTRODUCTION

The Cloud Design Guidance Annex describes software architecture patterns or reference architecture(s). The goal is to orient the reader to service offerings to develop, run, and manage applications within the various cloud environments. The information is to foster optimization of application design for the activity's architecture characteristics and type of workload. The architecture concepts used are defined below:

- **Reference Model** – is a model of something that embodies the basic goals or ideas, that can be used as a reference for various purposes. It is also useful for pulling together a bird's-eye view of a domain or problem space. There are two reference models are particularly important: the [Joint Architecture Reference Model \(JARM\)](#) and the reference models described by the NIST cloud computing service models¹. This annex focuses on the IaaS and PaaS service models. A reference model spawns one or more reference architectures.
- **Reference Architecture** - provides a template solution for an architecture for a particular domain. It's the high-level functional "blueprint" for a service or offering. It helps the designer see where they fit within the architecture. At times, we have referred to this as a "architecture pattern" and will use the two terms interchangeably in this document. A reference architecture spawns one or more reference implementations.
- **Reference Implementation** - is an actual architecture implementation; where the components are physically integrated and can serve as an exemplar for a type of service. A reference implementation is intended to follow the *consistent pattern* to maximize consistency in design of the enterprise services provided in the reference architecture.

NGA's priority in cloud design is to change the way that GEOINT capabilities are developed for timely mission use. Traditionally, developers were responsible for the entire stack, from infrastructure to applications (Figure 1). This approach required development time and resources to be shared across all levels of the stack.

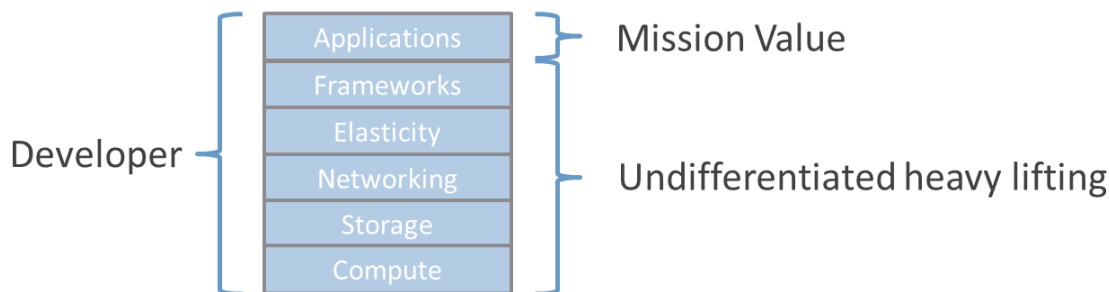


Figure 1: Traditional Development Approach - Developers Manage the Entire Stack

In the cloud environment, it became clear that developers would provide more value if their focus was further up the stack, and the provisioning/maintaining of the lower levels was offloaded to cloud infrastructure and platform providers. The first step on the transformation journey began

¹ As defined in [NIST SP 800-145](#), "The NIST Definition of Cloud Computing", September 2011

with ASP/ISP (Figure 2). Networking, storage, and compute were consolidated across the enterprise and made available to developers through an ISP. While this was a good first step, the cloud went a step further by building on the abstraction of infrastructure and adding elasticity, which gave the developer self-service provisioning on demand. This concept is known as the Infrastructure-as-a-Service (IaaS) cloud reference model.

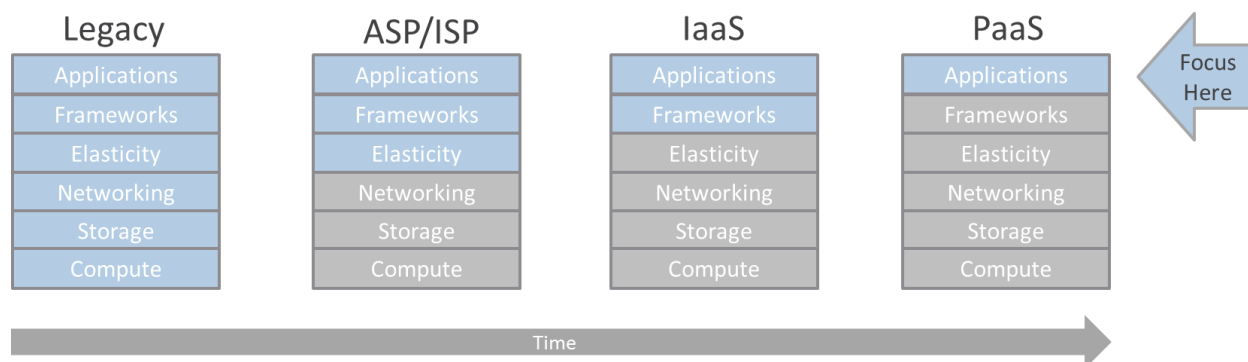


Figure 2: Cloud Reference Models: The Development Transformation Journey

Industry continued to push development further up the stack with an abstraction known as Platform-as-a-Service (PaaS). This reference model builds upon the IaaS model and adds software frameworks, middleware and container orchestration. As service utilization moves further up the stack, we are able to inherit more of the embedded services provided by the underlying service layers, thus enabling the developer's focus on application development and maximizing mission value.

PaaS can provide the most benefits, however, some workloads are better suited for IaaS given certain scenarios. Within PaaS itself, there are a number of industry offerings to address employment of different types of services. Scenarios will aid in understanding which offering is appropriate for a particular type of workload.

While the NIST cloud computing service models are useful for broad classification, it is important to note that the various vendors in the cloud industry have expanded their offerings, and it is common for a vendor to have offerings that span multiple NIST service models.

2.0 GUIDANCE

2.1 Design White Papers

Design white papers are published and updated to reflect the dynamics of the industry's cloud vendors expanding service offerings and capabilities. These white papers are published to the web and are in the [Cloud Guidance v5.2 directory](#).

Note that additional service offerings will be made available as new tools are deployed in the various clouds (e.g., AWS Elastic Map Reduce, OpenShift, DataBricks). Service availability is subject to government approval by service providers depending on the service demand and workload being addressed.

2.2 General Guidance

General guidance applies regardless of which reference architecture an entity chooses. In subsequent sections, specific guidance pertaining only to specific reference architectures/architecture patterns will be presented.

Guidance: *Activities shall design their software architecture to be modular in nature, leveraging existing services where possible.*

Guidance: *Best practices should include:*

- *Maximize service portability and access.*

Interfaces should be designed such that they can be used by multiple consumers. If there is only one possible consumer for a given service, this could be a clue that the service should be modified for greater access. Avoid point-to-point solutions, unless under restricted security controls.

- *Utilize a combination of technologies most appropriate for the particular characteristics of the activity workload(s).*

Custom code for long-running services may be best suited for a Cloud-Foundry-type PaaS + backing services, while big data analytics may be better-suited for something like Mesos/DCOS or the Databricks managed service. COTS-heavy applications may need to simply deploy onto IaaS.

- *Initiate Application Programming Interface (API) design as one of the earliest tasks in design efforts.*

APIs are a primary mechanism by which both internal and external customers will interact with a given service. API's should be designed so as to hide the implementation details of a given database or function while providing the ability for customers to interact with data and functions in a controlled fashion.

- *Document API's in OpenAPI format.*

By following the OpenAPI specification, activities enable improved automation and validation of APIs against their published specification, and have an explicit “build to” interface that can be shared with consumers.

- *Avoid mixing security contexts behind a single interface, when possible.*

This simplifies security decisions and reduces application complexity, allowing faster iteration of “internal” application logic.

- *Implement mechanisms for enabling “observability”.*

Observability in this context refers to generating data that can be used to drive business and technical decisions, and to help troubleshoot and resolve application issues. Application logs are one method of providing observability, but other monitoring techniques can also be employed.

- *Implement methods to isolate the impacts of failures within the application architecture.*

This includes designing a level of resiliency into the implementation, as well as adopting the “circuit breaker” pattern to support graceful degradation of functionality.

- *Leverage existing services from IC ITE designated service providers or other enterprise service providers, when possible.*

If existing services provide only a partial solution, use the partial solution and work to fill the gap either by working with the service provider to deliver the needed additional functionality, or by building out the functionality in a way that others can leverage (either by consuming a new service or by deploying the new code within their footprint).

2.3 IAAS Architecture Pattern (Reference Model)

Guidance: The IaaS pattern should be used in the following scenarios:

- Existing monolithic systems that are not easily refactored to be cloud native.
- Existing systems with significant COTS components that are not cloud native and are not suitable for running on the approved platforms.

In those scenarios addressed above, or where there is a need to minimize refactoring efforts due to programmatic constraints, this is an acceptable pattern to use. However, it bears repeating that NGA's intent is to move entities as far up the stack as possible, so the utilities gained in other architecture patterns are preferred over this minimalist approach.

Guidance: For those entities following the IaaS pattern, the following guidance should be applied:

- Utilize approved Virtual Machine Images (VMI's) that are security hardened, with Security Technical Implementation Guides (STIGs) applied, and include integrated base services.
- Take advantage of Cloud Services provided evaluation and monitoring tools (e.g., Trusted Advisor, CloudWatch) to maximize system and service resource utilization.

The IaaS pattern results in an application consuming IC ITE cloud services directly with basic NGA [services for the cloud environments](#). These cloud services provide on-demand provisioning for compute, storage, and networking resources. Entities using these services are responsible for operating systems and applications. AWS, as the current IaaS provider for the IC, has provided reference architectures that describe how their services can be used to handle various types of workloads. **Error! Reference source not found.** shows a reference architecture for the batch processing workflow. Others can be found on the [AWS Architecture Center](#) website.

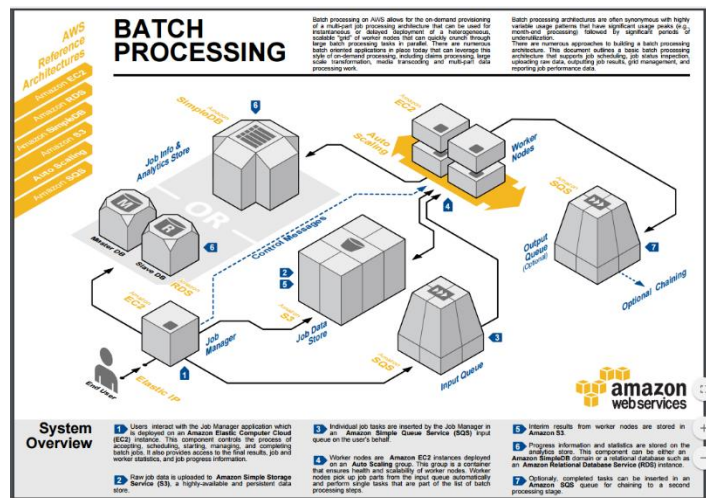


Figure 3: Sample Reference Architecture

Entities leveraging the IaaS Pattern are able to inherit security controls that are satisfied by the cloud service provider; more details are provided in Annex I: Cloud Security.

For those entities following the IaaS pattern, the following basic NGA services are provided:

- Enterprise Services (e.g., A&A, domain services, compliance tools, utilization tools)
- DevOps tools from both IC ITE and NGA.

The IaaS pattern follows the assisted service model for implementation using engineers and administrators working with entities directly.

2.4 PaaS Architecture Pattern (Reference Model)

Guidance: *The PaaS pattern should be used in the following scenarios:*

- *Activities following modern cloud-native software practices, such as 12-factor app principles² and micro-services.*
- *Activities that expect to be updated regularly and can benefit from an efficient code deployment eco-system.*
- *Activities directed to refactor critical enterprise backing services as Platform-hosted offerings (such as PostGIS).*

The PaaS reference model is a level of abstraction on top of IaaS that provides an environment for deployment, scaling, and management of applications. Whereas, in the IaaS architecture pattern the developer has to manage operating systems, load balancers, subnets, and other related items; the PaaS offering manages IaaS components for the developer so that they can focus more attention on delivering mission value.

The PaaS architecture pattern provides a number of benefits for entities:

- Reduces time spent on infrastructure development and management
- Increases available time for enhanced code quality and more development capacity
- Built-in resilience – platform handles failures of underlying infrastructure
- Can abstract different IaaS providers, avoiding lock-in
- Applications inherit more security controls than when using the IaaS architecture pattern, which simplifies and expedites security accreditation
- Integrates with the DevOps pipeline.

For those entities following the PaaS pattern, the following resources are provided:

- Enterprise services (A&A, domain services, compliance tools, utilization tools)
- DevOps tools from both IC ITE and NGA.

The PaaS pattern follows the assisted service model for implementation: engineers and administrators work with entities directly.

Guidance: *For those activities deploying into Pivotal Cloud Foundry (PCF), the following guidance should be applied:*

- *Applications that follow 12 factor app principles are best suited for hosting within PCF.*
- *Developers should use the PCF development environment for development and testing activities; then, once capabilities reach an operational state use the PCF operations environment.*
- *Developers should make every effort to utilize the build packs that are provided with PCF, and should not apply custom build packs unless absolutely necessary.*

² [12-factor app principles](#)

Pivotal Cloud Foundry (PCF) is a structured, opinionated³ platform that is designed to run cloud native applications. Figure 4 shows the PCF Reference Architecture. Developers “push” application software to the platform; the platform utilizes build packs to package the software with appropriate middleware (such as software frameworks), containerizes, and then deploys into the platform runtime. The platform runtime takes care of routing, service discovery, container scheduling, and configuration. Backing services are available in the PCF Marketplace to provide capabilities like databases and object storage that can be bound to applications.

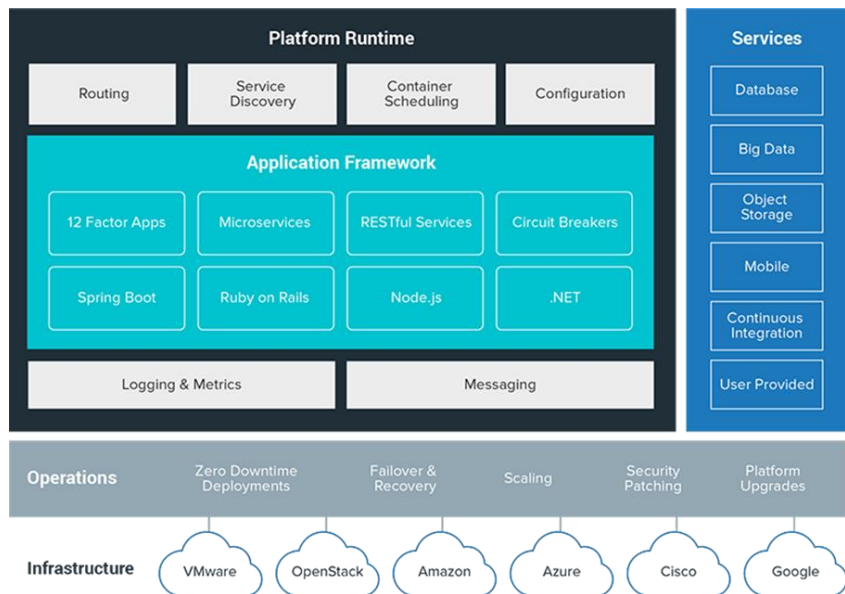


Figure 4: PCF Reference Architecture

From an operations perspective, the PCF platform provides capabilities that allow scaling on demand, simplified security patching, automated failover and recovery, and deployments that require no downtime. PCF also runs on a variety of different IaaS providers.

2.5 NDAAS Architecture Pattern (Reference Model)

Guidance: The NDAAS pattern should be used in the following scenarios:

- Activities providing multi-region enterprise services with operations requiring datacenter-to-datacenter cluster analytics.
- Activities that need to distribute analytics where the data resides.
- Activities that are able to package and deploy their applications in containers (e.g., Docker).
- Activities following modern cloud-native software practices, such as 12-factor app principles and open-standard micro-services.

Guidance: The NDAAS architecture pattern should be employed when the workflow requires running distributed applications at a very large scale and requires an extensible architecture to support a multi-region operational mission footprint.

The [NSG Data Analytic Architecture Service \(NDAAS\)](#) is a PaaS offering that provides a data-centric environment for geospatial services for the community. This approach provides a number of benefits for entities selecting this reference architecture:

- Deploy, operate and upgrade complex technologies such as Spark, Kafka and Cassandra with a single command.

³ Platforms are “opinionated” because they make specific assumptions and optimizations to remove configuration complexity from the user – Cloud Foundry - The Cloud-Native Platform, by Duncan C. E. Winn.

- Maintain an infrastructure with high availability, automatic failure detection and recovery and non-disruptive upgrades.
- Integrates with Active Directory or LDAP for identity authentication and authorization and provide role-based access control and user auditing.
- Easily integrate with release automation and Continuous Integration and Continuous Delivery (CI/CD) tools (e.g., security testing) to accelerate your software release lifecycle from development to production.

For those entities following the NDAAS pattern, the following containerization and cluster management services are provided:

- Data Center Operating System (DCOS) PaaS with Docker, Apache Mesos, Marathon and NGA Scale.
- Enabling applications, including Cassandra and Jenkins.

The NDAAS provides greater focus and attention to developing geospatial mission applications, given the emerging platform-based capabilities realized through self-managing, scalable horizontal service architectures. The architecture design provides ready mission responsiveness to implement timely geospatial applications through the integration of shared enterprise capabilities and uniform security management with DevOps flexibility.

NDAAS employs key technology items (e.g., containers, micro-services, elasticity, cluster managers) operating in an open, scalable, integrated horizontal architecture. In this architecture, each of the functional flows instantiate a cluster of resources via containerization to enable the processing of multiple, independent activities simultaneously. This shift from vertical dedicated architectures and resource allocation to a horizontal shared architecture is a priority in performing NGA's massive data analytics.